

Data Engineering and Analytics Project Report

Team Name: Tourism

Member 1: Malak Abdelghani (0231497)

Member 2: Nour Swaid (0234794)

Member 3: Ayat Shehadeh (0248473)

May 23, 2026

Abstract

This project analyzes Airbnb rental data using data engineering techniques, our goal is to prepare the data for specific analysis which are mainly about finding the feature or categories that affect the customer and business decision. The work included several stages such as data acquisition, data cleaning, preprocessing, feature engineering, transformation, validation, and visualization.

1 Introduction

Since the online renting process has been so popular nowadays, a rental platform has been generated to help the customer find a rental place that suits their requirements the best. So, our Airbnb dataset has grown popular in online market in the short-term lodging rentals. Airbnb dataset contain prices, room types, rooms availability, customer review and others that are important features that help analyzing the data to support the business and customer decisions.

However, the missing values, duplicate records, invalid entries, and noisy attributes are common in raw Airbnb datasets, which lower data reliability and analytical quality. Therefore, data engineers are required to perform efficient data analysis.

Our project includes data engineering process starting from data acquisition, inspection, cleaning, preprocessing, feature engineering, analysis and data visualization. The main goals are finding the factors that influence Airbnb listing revenue the most, which neighborhoods offer the highest revenue potential, and what features are associated with higher demand (booking).

2 Problem Definition and Motivation

The purpose of this project is to identify an important problem with the way many landlords manage their housing units and use data analysis to help solve this problem. Although there is a lot of information available in the “AirBnB Open Data” dataset in New York city, most of this information is presented in ways that make it difficult to understand. This dataset contains missing values, inconsistencies, and corrupted text. There are no columns in the data that report what amount of money properties generate. Most of the time instead of getting exact dollar amounts based on clear profitability, you get a gap. It is very difficult to gain any level of understanding into your data until all the bad data is corrected. If you don't have good data on actual income levels, then you cannot draw logical conclusions about your data. To correct these issues we need to transform this initial dirty data (like neighborhood groupings, usage restrictions (number of nights minimum, availability per year), etc.) into clean data that works well for analytical purposes. Once transformed into clean usable data, the output needs to be organized, merged, and ready for reporting and creating new features. From a practical

standpoint this work is significant due to the fact that real world problems exist. Despite being regulated tighter and tighter each year, there are still individuals who continue to enter into New York City’s overcrowded short-term rental market. As such, finding the best place to rent out property remains a guessing game. Some have started developing predictive models that assist in decision-making (such as price or booking limits) rather than reactively selecting where to rent out property. When large amounts of unorganized, disordered, and messy information is cleaned and transformed correctly, hidden trends begin to reveal themselves. Dollars move better when numbers tell a more clearly defined story. Decisions transition from gut feelings toward decisions that are more predictable.

3 Background and Selected Data Engineering Approach

Before we could create an ingest system that is reliable and consistent, we had to set up an ingestion process using Python/Pandas that utilizes a structured, batch-based architecture. Loading a CSV file seems like a straightforward task; however, when dealing with real-world data, this is one of many ways the task can be broken.

3.1 Ingestion Phase and Fault-Tolerance

During the transformation phase, we encountered a significant technical barrier. The dataset contained severe text-based formatting anomalies within columns, due to the fact that users wrote their own apartment descriptions and rules containing unclosed quotations and special characters, which caused structural confusion for the analytical calculations.

By deploying the following technical strategy, we managed to ensure a fault-tolerant ingestion process:

- **Parser Engine Switch:** Our default C optimized parser has been switched to the much more strong `python` execution engine.

3.2 Feature Engineering and Demand Proxies

Once we ensured data ingestion was safe, we then moved on to augmenting our dataset by creating new features that will help fill the gaps in current knowledge for using in our analytical models:

- **Demand Proxy Modeling:** Utilization of an alternate measure to occupancy for demand modelling because we are not able to get the actual occupancy rates. We use the `reviews_per_month` field as a credible statistical proxy to measure demand in the market.
- **Temporal Feature Engineering:** We found that using the static variable `construction_year` for modeling did not provide much analytical value. Therefore, we leveraged our project’s baseline year of 2022 to compute a dynamic attribute, `listing_age`, applying the following temporal logic:

$$\text{Listing Age} = 2022 - \text{Construction Year} \tag{1}$$

- **Non-Linear Trend Analysis:** By deriving the asset age, we were able to segment the listings into distinct age brackets, allowing us to capture and analyze complex, non-linear market trends that would otherwise remain hidden when observing only raw numbers.

4 Data Engineering Workflow and Pipeline Modelling

In this project, a structured data engineering pipeline was followed to transform raw Airbnb data from messy and inconsistent data into a clean and structured dataset suitable for analysis. The

workflow was designed to ensure the data became reliable, consistent, and capable of supporting business decision-making.

The pipeline followed the sequence below:

Raw Data → Data Ingestion → Data Cleaning → Transformation → Feature Engineering → Analysis → Visualization

This pipeline is reproducible because all transformations are deterministic and applied in a fixed order. The same raw input will produce identical cleaned output. The workflow is also extensible since additional cleaning stages or new data sources can be integrated without modifying the existing workflow structure.

4.1 Data Quality Problems and Solutions

The raw dataset contained several data quality issues that required preprocessing before analysis could be performed.

4.1.1 Removing Irrelevant Features

Several features did not contribute meaningful analytical value and were therefore removed. These included identifier columns and highly inconsistent free-text columns that could not be standardized effectively. Additionally, some features contained no analytical variation. For example, all listings belonged to the same country, making the country feature unnecessary for analysis.

```
1 data.drop(columns=['id', 'host_id', 'NAME',  
2 'host_name', 'country', 'country code'],  
3 inplace=True)
```

The dataset also contained formatting inconsistencies in numerical columns stored as strings with symbols such as \$, %, commas, and spaces. These inconsistencies were standardized before converting the columns into numerical format.

```
1  
2 data["service fee"] = (  
3     data["service fee"]  
4     .str.replace("%", "", regex=False)  
5     .str.replace("$", "", regex=False)  
6     .str.replace(",", "", regex=False) )  
7  
8 data["service fee"] = pd.to_numeric(  
9     data["service fee"],  
10    errors="coerce")
```

4.1.2 Handling Invalid Values

Some features contained logically invalid values. For example, the *minimum nights* feature contained negative values, which are impossible in the context of property rentals. Similarly, the *availability 365* feature also contained negative values. These invalid observations were removed.

```
1 data = data[data['minimum nights'] >= 0]  
2 data = data[data['availability 365'] >= 0]
```

Additionally, around 2700 observations in the *availability 365* feature exceeded the logical upper limit of 365 days. Removing these rows would have resulted in losing nearly 3% of

the dataset, so clipping was applied to preserve the observations while maintaining the logical constraint.

```
1 data['availability 365'] = np.where(  
2     data['availability 365'] > 365,  
3     365,  
4     data['availability 365']  
5 )
```

4.1.3 Handling Missing Values

Multiple columns contained missing data, so different imputation strategies were applied depending on the feature type and the reason for missingness.

One feature contained approximately 99.99% missing values with only two valid observations across the dataset. Since it did not provide meaningful analytical value, it was removed entirely.

```
1 data.drop(columns=['license'], inplace=True)
```

Some geographical coordinate-related features also contained missing values. Since random imputation could generate logically impossible coordinates, rows with missing coordinate values were removed instead of imputed.

For numerical features, missing values were filled using the median because it is less sensitive to outliers and skewed distributions. Categorical features were filled using the mode.

A special case existed for the *reviews per month* feature. The assumption was that some missing values represented listings with no reviews. However, replacing all missing values directly with zero would create contradictions with the *number of reviews* feature. Therefore, conditional imputation was applied.

```
1 data['reviews per month'] = np.where(  
2     (data['reviews per month'].isna()) &  
3     (data['number of reviews'] == 0),  
4     0,  
5     data['reviews per month']  
6 )  
7  
8 remaining_median = data['reviews per month'].median()  
9  
10 data['reviews per month'] = (  
11     data['reviews per month']  
12     .fillna(remaining_median)  
13 )
```

4.1.4 Data Transformation

Several features were transformed to improve their analytical value.

For example, the *house rules* feature originally contained free-text rental policy descriptions. Since the project did not involve natural language processing, the raw text itself was not directly useful for analysis. Therefore, the feature was transformed into a binary indicator showing whether a listing had house rules or not.

```
1 data["has_house_rules"] = (  
2     data["house_rules"].notnull().astype(int)  
3 )  
4  
5 data.drop(columns=['house_rules'], inplace=True)
```

Later analysis showed that listings with house rules were associated with lower estimated revenue on average, suggesting a possible relationship between stricter policies and booking activity.

4.1.5 Duplication Removal

Duplicate observations were removed to prevent inflated statistics and biased analytical results.

```
1 data.drop_duplicates(inplace=True)
```

4.1.6 Outlier Handling

Outlier handling was performed using multiple techniques depending on the feature.

A boxplot was first used to visually identify extreme values and understand the distribution of key numerical variables.

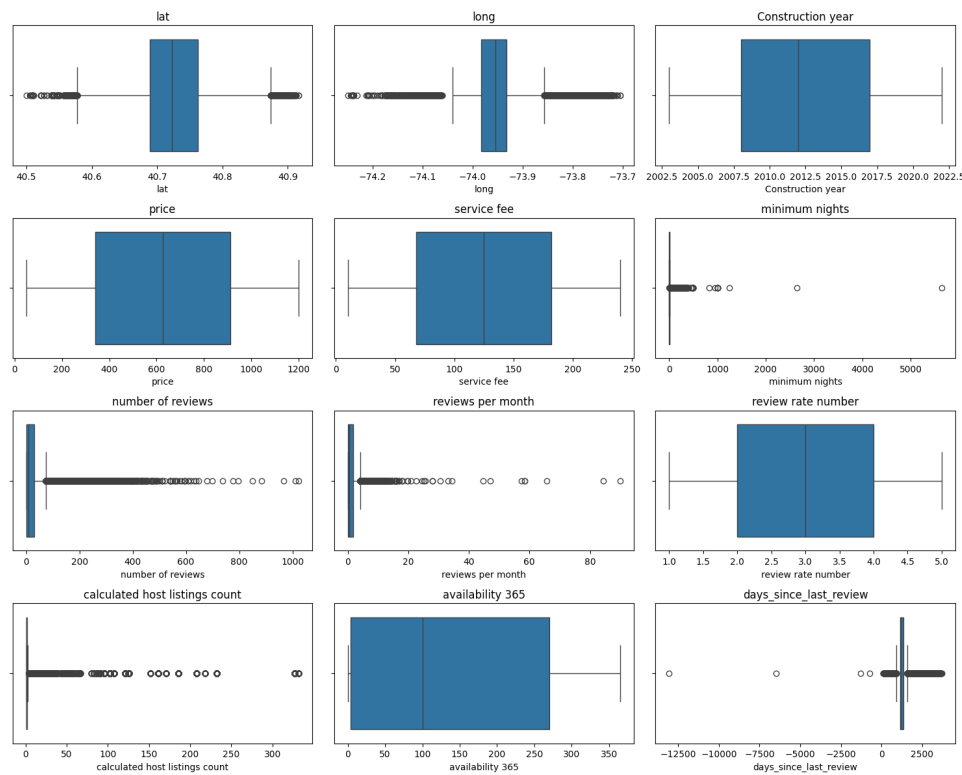


Figure 1: Boxplot for outlier detection

Based on this analysis, different treatment strategies were applied depending on the nature of each feature.

For example, the *minimum nights* feature contained extremely high values. While technically possible, values exceeding one year are unrealistic in practical rental scenarios. Therefore, clipping was applied.

```
1 data['minimum nights'] = (  
2     data['minimum nights'].clip(upper=365)  
3 )
```

For features with heavily right-skewed distributions, logarithmic transformations were added as separate columns while preserving the original features for interpretability.

```

1 skewed_cols = [
2     'number of reviews',
3     'reviews per month',
4     'calculated host listings count'
5 ]
6
7 for col in skewed_cols:
8     data[f'{col}_log'] = np.log1p(data[col])

```

4.2 Feature Engineering

Feature engineering was performed by both transforming existing features and creating entirely new features.

For example, the *Construction year* feature was transformed into a listing age feature, and the *last review* feature was converted into a *days since last review* feature.

Additional features were created to capture relationships between variables. Since actual booking revenue was unavailable, estimated monthly revenue was approximated using a proxy metric based on price and review activity.

```

1 data['est_monthly_revenue'] = (
2     data['price'] * data['reviews per month']
3 )

```

4.3 Visualization

Visualization techniques were used to analyze relationships between features and estimated revenue.

Several visualizations were created, including:

- Bar charts showing feature impact on estimated revenue
- Correlation analysis between features and demand
- Feature impact visualizations showing how specific feature values influence demand

These visualizations helped identify important business insights and supported interpretation of preprocessing decisions and engineered features.

1. Visualization showing feature impact on estimated revenue

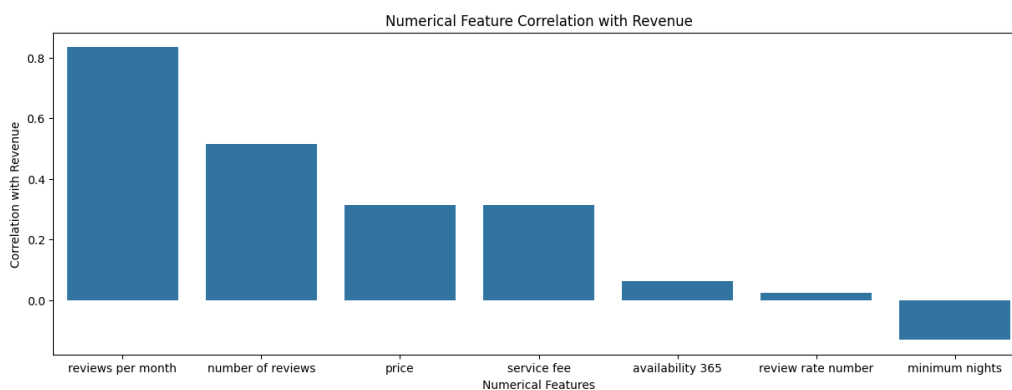


Figure 2: Feature impact on estimated revenue

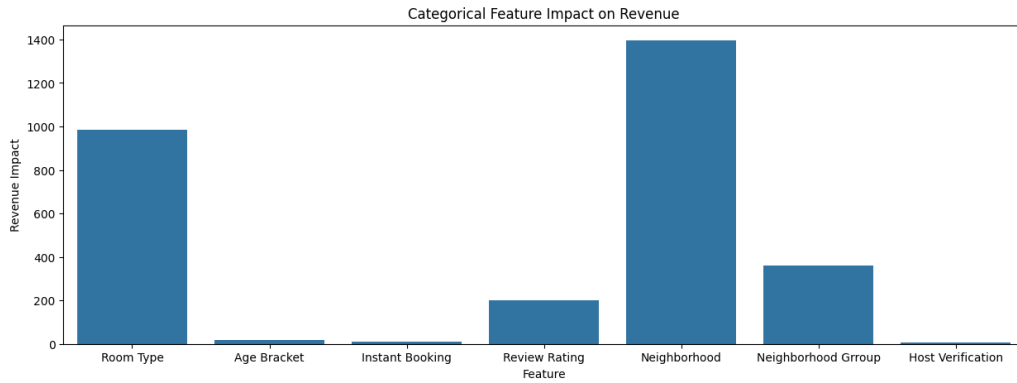


Figure 3: Feature impact on estimated revenue

2. Correlation analysis between features and demand

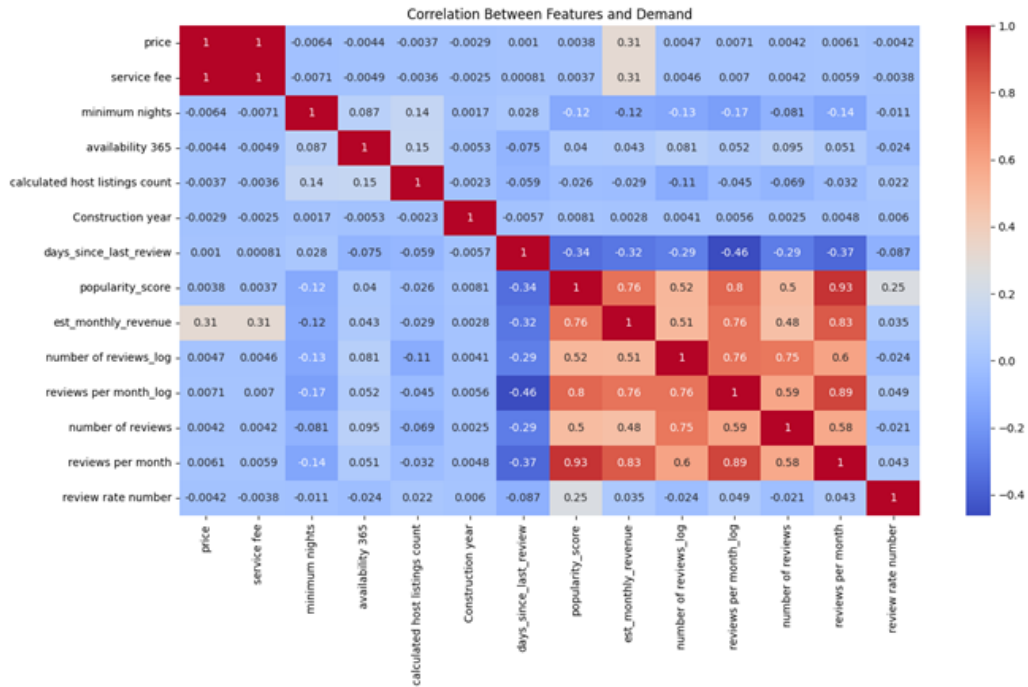


Figure 4: Correlation between features and demand

3. Plots showing which feature values influence demand the most

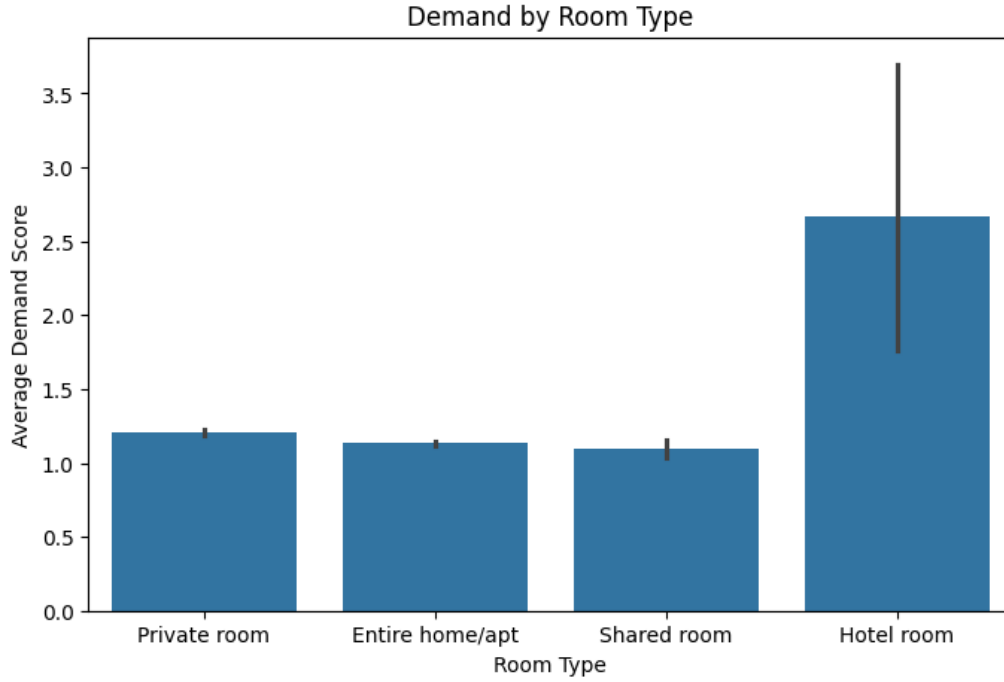


Figure 5: Feature values influence on demand

5 Data Modelling and System Design

To convert the real-world short-term rental issue into an automated data structure, the project domain was thoroughly modeled and separated into several discrete logical pieces. The entire structure is built to ensure quality, uniformity, and correct separation of raw data and analytic data.

5.1 Logical Domain Entities & Attribute Values

The features from the dataset were categorized into three main logical entities to model the business domain and correlate the features with their operational functions:

- **Entity Host:** Provides the operational profile and identification of the property owner. Major attributes consist of: `host_name`, `host_identity_verified`, the existing baseline feature calculated `host listings count`, and our generated categorical field `host_type` (Individual vs. Professional).
- **Entity Listing/Property:** means it holds the information of a physical or spatial and geographic property. The primary characteristics include: `id` (functioning as the singular primary identifier for each listing), `name`, `neighbourhood_group`, `neighbourhood`, `room_type`, and the created numerical field `listing_age`.
- **Constraint/Operational/Fiscal Entity:** The entity that holds the constraints, prices, performance, Metrics and transactions related to bookings and revenue generated from clients. Major attributes consist of: `price`, `service_fee`, `minimum_nights`, `availability_365`, and `reviews_per_month`.

5.2 Analytical Tables and Created Derivative Fields

Derivative fields were developed in addition to the original fields that exist in the source data. This was done using both explicit technical logic as well as feature engineering to fill information

voids identified in the raw data.

- **Derivative Fields:**

- **Estimated Monthly Revenue:** is one derivative field that has been added. It is a measure of monthly financial performance that was developed through use of an operational functional proxy; $\text{Price} \times \text{Reviews Per Month}$.
- **Listing Age:** determines how old the real estate listing is.
- **Host Type:** has been generated using a conditional rule based on the lambda logic that uses a `calculated host listings count`.

- **Analytical Aggregation Tables:** The pipeline performs explicit processing instead of using large, flat transactional tables. The aggregation operations that create and maintain the analytics are carried out by Python's `groupby()`. With summarization, huge counts of records can be efficaciously compressed into four major DataFrames for tactical choices:

- **Revenue Location Table:** Used to organize revenue data by geographic area to rank neighborhoods with high yield.
- **Market Segmentation Table:** Categorizes and aggregates pricing, demand, and revenue statistics across different `room_type` categories.
- **Host Performance Table:** According to the empirical averages, a professional host generates around \$685.19 in estimated monthly revenue, while an individual host generates around \$397.27.
- **Feature Correlation Matrix:** This calculates total expected revenue as a function of a minimum stay operating constraint.

5.3 Schema Design and System Data Layers

A multi-tiered version of the 3 tier architecture is implemented within our system's architecture. An advantage to using a three-tier structure for our system design is to provide a way to implement strict rules on data quality and attribute definitions:

1. **Raw Data Layer (Source Table):** The first place we drop the unedited CSV file into, it contains all of the structural irregularities, non-existent attributes, inconsistent text based pricing, and potential parser error's from issues like broken lines with unclosed quote marks.
2. **Cleaned Data Layer (Processed Table):** After running an extensive python parser against the original CSV file, this is typically the location the cleaned up data will reside. Once parsed, this layer consists of only those rows of the CSV where the parse was successful. Rows containing specific errors are excluded, each column of the CSV containing financial data in textual format is converted to a numerical field (float), and a standardization method is applied to missing value's.
3. **Analytical Data Layer (Presentation Tables):** Since the previous layer has standardized the data to a common schema, the analytical data layer provides complete separation of all core business logic from raw anomalous data; thereby, providing all downstream investment analytics with consistent and accurate high quality data.

6 Illustrative Figure Usage

In this section insights and decision making outcomes gained from the project are discussed.

6.1 Demand by Minimum Night

The analysis of stay category and minimum nights demand reveals a clear pattern. Most listings focus on short stays (1–3 nights) and weekly stays (4–7 nights), while long-term stays have fewer listings on the platform. This indicates that both hosts and guests prioritize the short-term market.

The analysis also shows that as minimum nights requirements increase, demand and revenue decline. Guest demand is consistently directed toward shorter stays, aligning with typical travel patterns and booking behavior.

Decision Insight Gained

To maximize demand and revenue, hosts should set minimum nights requirements to 1–3 nights and focus on short- to weekly-stay categories, avoiding longer minimums that consistently reduce guest demand.

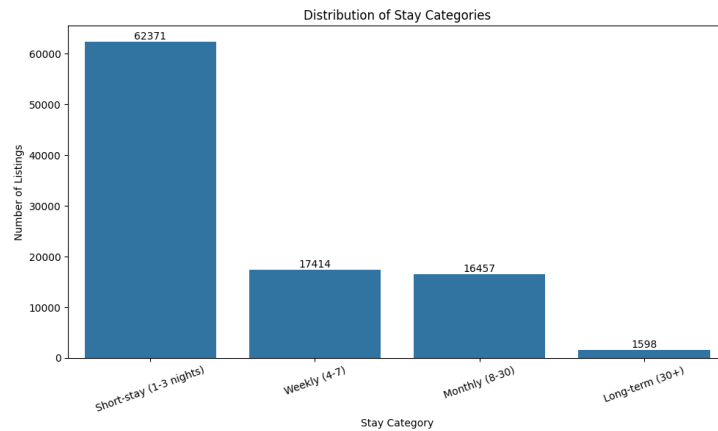


Figure 6: Distribution of stay categories (short, weekly, long-term)

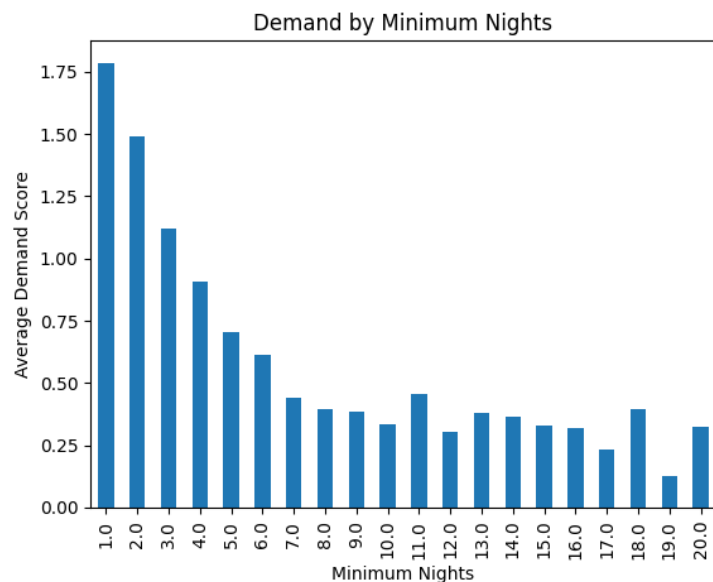


Figure 7: Minimum nights impact on Demand

6.2 House Rules Impact on Revenue

Listing without house rules captures 53% of estimated revenue

The chart indicates that properties with no house rules generate a large share of platform revenue. Suggesting that guests prefer fewer strict rules. Therefore, Hosts should evaluate whether their current house rules are necessary or if reducing them could improve booking rates and overall revenue.

Decision making for hosts:

- Review rules necessity
- Reduce booking friction

House Rules Impact on Estimated Monthly Revenue

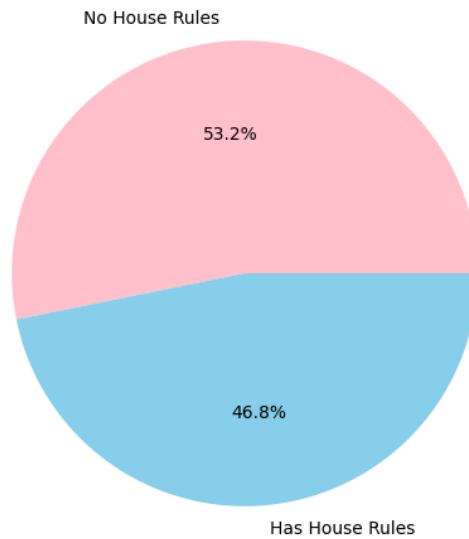


Figure 8: House rules impact on revenue share

6.3 Cancellation Policy

Contrary to the assumption that strict cancellation policies negatively affect booking, analysis on the data yields the opposite finding. Flexible, moderate and strict cancellation policies yield nearly identical average demand score. This means host can choose their preferred policy based on their listing type without worrying about harming the booking demand. However, flexible policies may provide a slight edge in highly competitive markets.

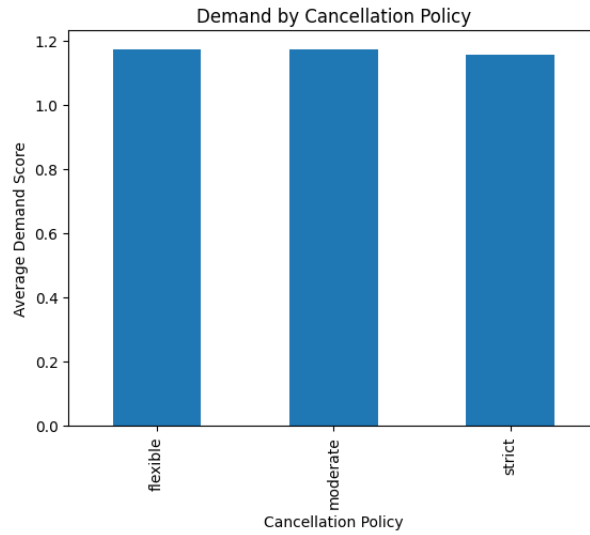


Figure 9: Cancellation policy impact on demand

6.4 Demand by Instant Booking

Similar to the cancellation policy, instant booking yields nearly the same average demand results. While instant booking can improve guest experience and reduces host workload, it does not appear to significantly drive demand on its own. Hosts should decide based on their preferences rather than expecting an automatic boost from enabling that feature alone.

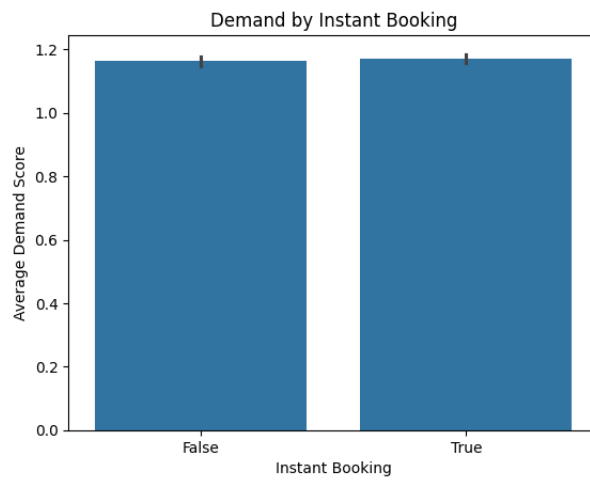


Figure 10: Instant booking impact on demand

6.5 Demand by Neighborhood Group

Insights: The top 10 neighborhoods by average revenue (e.g., Chelsea, Staten Island at 3,150) are all in Staten Island and Queens, yet the borough-level chart shows Manhattan with the lowest average revenue ($\sim 300\text{--}400$) and Staten Island the highest ($\sim 1,000+$). This suggests Manhattan has many low-revenue areas that drag down its average, while Staten Island has both high performing pockets and solid overall performance. The large gap between top neighborhoods and borough averages indicates revenue is highly concentrated in specific areas rather than evenly distributed across boroughs.

Decision-making :

- **Prioritize Staten Island** for investment, sales, and field operations, especially Chelsea, Huguenot, and Huguenot.
- **Target Queens as a secondary market**, focusing on Springfield Gardens and Glen Oaks.
- **Avoid broad spending in Manhattan**; instead, micro-target specific high-revenue pockets.

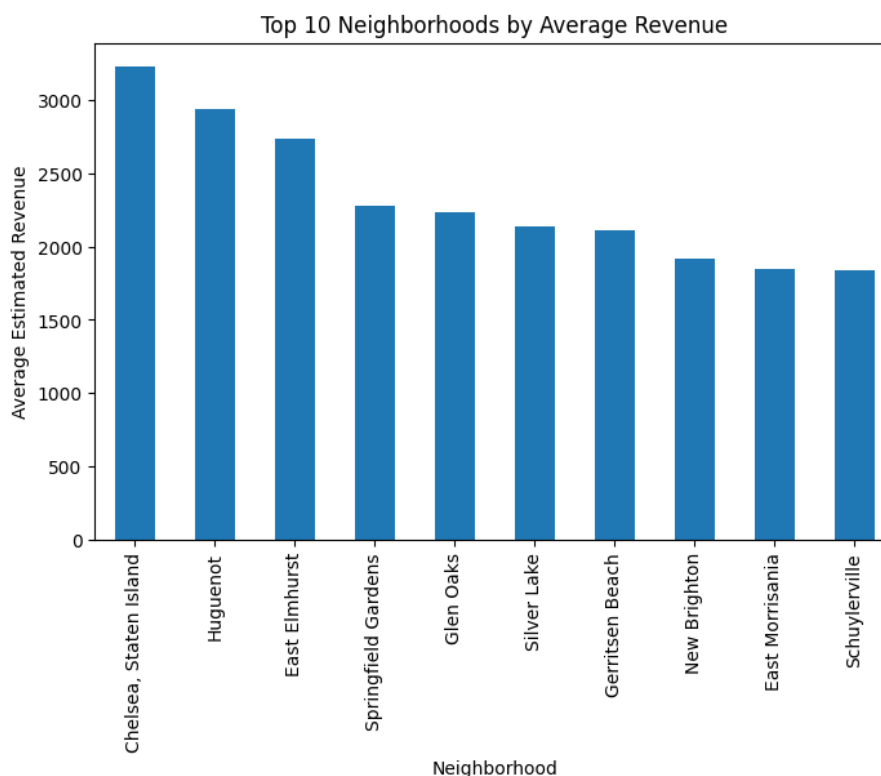


Figure 11: Neighborhood distribution revenue analysis

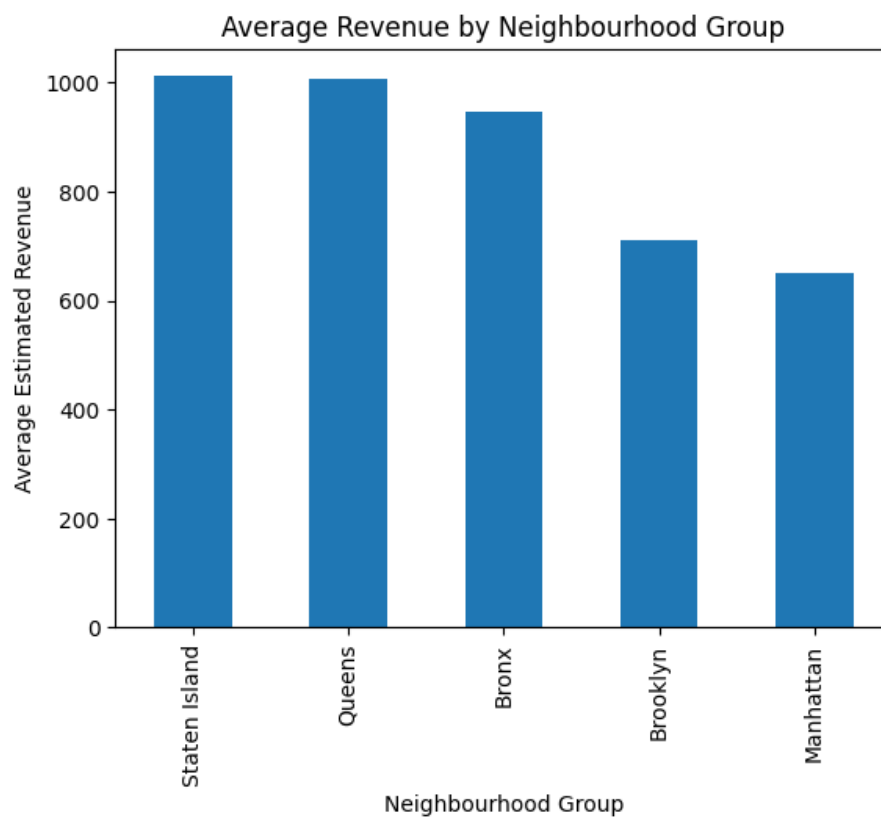


Figure 12: Neighborhood group revenue analysis

6.6 Demand by Room Type

Insight: Hotel rooms generate the highest demand, likely because they offer the best balance of price, convenience, and familiarity for most travelers, but also the **highest variation** in that demand. Private rooms rank second. Entire homes and shared rooms drive the least demand, but not because they are inferior—they serve niche segments: shared rooms appeal to budget-conscious social travelers seeking interaction, while entire homes target those wanting privacy and a localized, relaxing experience.

Decision-making :

- Double down on hotel room inventory or partnerships – highest demand means fastest turnover and stable revenue.
- Do not cut entire homes or shared rooms – they serve profitable niches; instead, market them specifically (e.g., "perfect for groups" or "meet other travelers").
- Tailor marketing messages:
 - Shared rooms → highlight social vibe, common areas, events.
 - Entire homes → highlight privacy, kitchen, laundry, "live like a local."

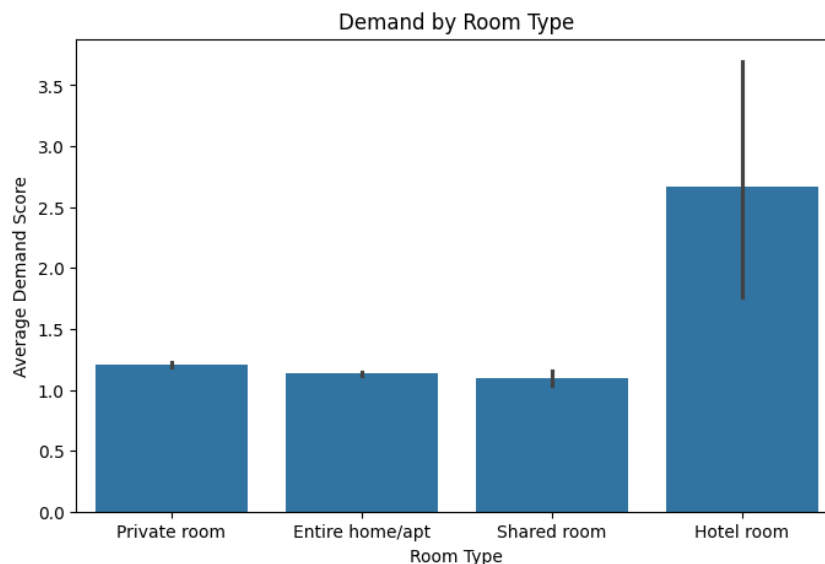


Figure 13: Room type demand analysis

6.7 Reviews Per Month and Demand

The bar chart (left) shows a staircase effect:

- Inactive listings (0 reviews/month) → \$0 revenue
- Low activity (0–1/month) → \$224/month
- Moderate (1–3/month) → \$1,178/month
- High (3+/month) → \$2,914/month

The scatter plot (right) reinforces why price alone is misleading — listings at \$200/night with high review velocity (darker dots) often earn more than listings at \$800/night with low

activity (pale dots). The dark dots are scattered across all price ranges, not concentrated at the top.

The host decision this points to:

The number one priority for any host is not raising their price, not decorating, not getting verified — it's **staying booked consistently enough to generate 3+ reviews per month**. That means: keeping availability open, responding fast, pricing competitively to fill gaps, and actively asking guests to leave reviews. A host earning \$224/month who manages to double their booking frequency crosses into the \$1,178 tier will get more gain just from staying active, with zero change to their listing itself.

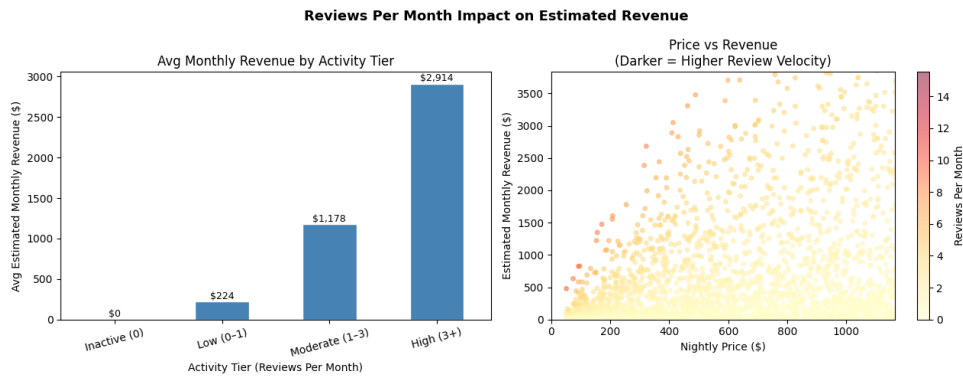


Figure 14: Reviews per month impact on estimated revenue

6.8 Summary of Feature Correlations

Charts below summarizes how each characteristic / feature influences the revenue.

Reviews per month leads at 0.83 correlation with revenue, nearly double the next strongest predictor, number of reviews at 0.52. Price and service fee both sit at 0.31, meaning a higher nightly price does lift revenue somewhat, but far less than booking frequency does. Review rate number (the star rating) barely registers at 0.03, confirming it has no real relationship with revenue. Minimum nights are the only negative predictor at -0.08 , meaning longer required stays actively suppress revenue rather than improve it.

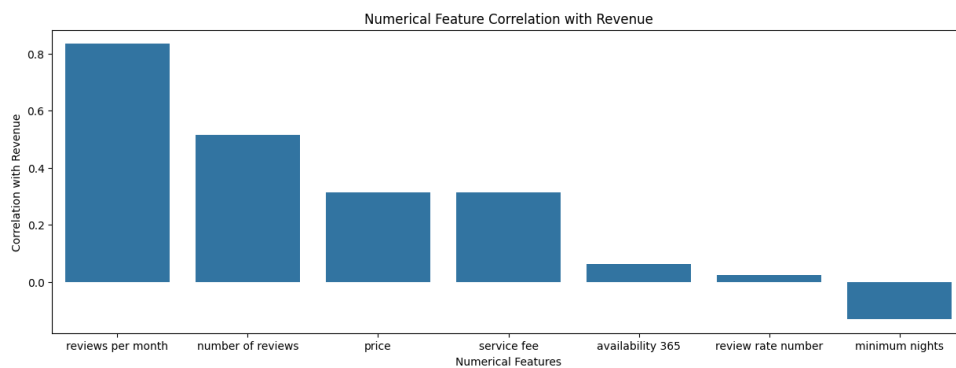


Figure 15: Correlation between features and revenue

For the categorical features, Room type, neighborhood and neighborhood group drive the revenue more than any other feature.

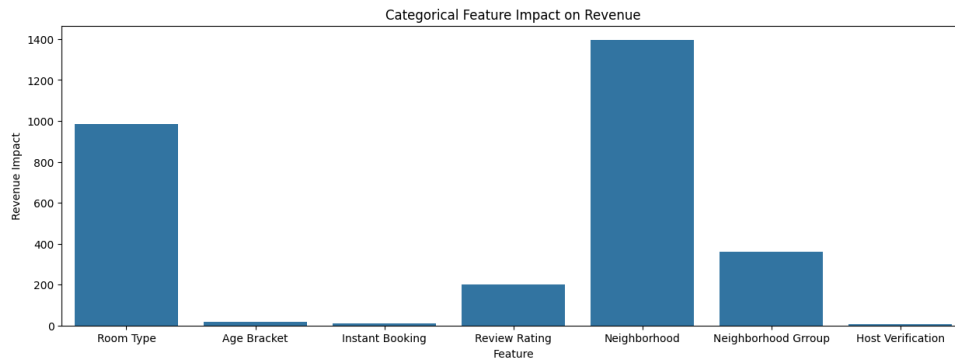


Figure 16: Categorical feature impact on revenue

7 Implementation

For the technical implementation, multiple tools were used to convert the data into a clean, structured format that can be used to gain insights and make decisions.

Main tools: Python, Pandas, Matplotlib, Seaborn, NumPy

7.1 Importing the Necessary Libraries

```
1 import pandas as pd
2 import numpy as np
3 import math
4 import matplotlib.pyplot as plt
5 import seaborn as sns
```

7.2 Data Inspection and Exploration

Explore data shape:

```
1 data.shape # -> (102599, 26)
```

Check data by first 5 rows:

```
1 data.head()
```

Explore the data features:

```
1 list(data.columns)
```

Map each column to its explanation: Easier to follow when multiple people work on the same project.

```
1 pd.DataFrame(list(zip(cols, col_meaning)), columns=['column', 'meaning'])
```

Data statistics and information, explain number of null values and data types:

```
1 data.describe()
2 data.info()
3 data.dtypes
```

Number of null values:

```
1 data.isnull().sum().sort_values()
```

7.3 Dropping Irrelevant Features

Dropping irrelevant features that do not contribute to analysis:

- id, host id: Unique identifiers (no analytical value)
- NAME, host name: Free text fields (inconsistent, hard to standardize)
- country, country code: All listings in US (no variation to analyze)

```
1 data.drop(columns=['id', 'host id', 'NAME', 'host name', 'country', 'country  
code'], inplace=True)
```

7.4 Fixing Inconsistent Format

```
1 data['neighbourhood group'] = data['neighbourhood group'].replace({  
2     'brookln': 'Brooklyn',  
3     'manhatan': 'Manhattan'  
4 })
```

Converting price into numerical format and removing strings from its value:

```
1 data["price"] = (  
2     data["price"]  
3     .str.replace("$", "", regex=False)  
4     .str.replace(",", "", regex=False)  
5     .str.replace(" ", "", regex=False)  
6 )  
7  
8 data["price"] = pd.to_numeric(data["price"], errors="coerce")  
9  
10 data["service fee"] = (  
11     data["service fee"]  
12     .str.replace("%", "", regex=False)  
13     .str.replace("$", "", regex=False)  
14     .str.replace(",", "", regex=False)  
15 )  
16  
17 data["service fee"] = pd.to_numeric(data["service fee"], errors="coerce"  
    )
```

7.5 Removing Wrong Entries

Minimum nights had invalid values below zero, which would contradict the meaning of the feature. It was about 7 entries so they were all dropped.

```
1 data = data[data['minimum nights'] >= 0]
```

Same for the availability 365 feature, it has values under zero which were dropped. As for values above 365, there were approximately 2,700 rows where availability_365 exceeded its theoretical maximum of 365.

Since this variable is strictly bounded between 0 and 365, these values represent data quality issues rather than valid extreme observations.

Removing these rows would result in a loss of nearly 3% of the dataset, which could negatively impact downstream analysis.

To preserve observations while enforcing logical constraints, capping at 365 was applied:

```

1 data['availability 365'] = np.where(
2     data['availability 365'] > 365,
3     365,
4     data['availability 365']
5 )

```

7.6 Handling Duplicate Values

To ensure data integrity:

```

1 data.drop_duplicates(inplace=True)

```

7.7 Handling Missing Values

Show percentage of missing values:

```

1 (data.isna().sum() / len(data) * 100).sort_values(ascending=False)

```

The license column had over 99.99% missing values — only 2 listings out of 102,000 had license information. At that level of sparsity, the column provides no actionable signal and was dropped entirely.

```

1 data.drop(columns=['license'], inplace=True)

```

The house rules feature contained approximately 51% missing values and consisted of unstructured textual data, which was not suitable for direct analysis. Since the text was inconsistent and not standardized, it was transformed into a binary indicator representing presence or absence of house rules.

Missing values were encoded as 0, assuming absence implies no specified house rules.

```

1 data["has_house_rules"] = data["house_rules"].notnull().astype(int)
2 data.drop(columns=['house_rules'], inplace=True)

```

For the number of reviews feature, it was filled with zero assuming that if it is missing then likely there were no reviews:

```

1 data["number of reviews"] = data["number of reviews"].fillna(0)

```

For the review per month feature, a special approach was followed. Direct imputation using zero or median for all missing values could lead to inconsistencies, such as non-zero review rates for listings with zero total reviews. So conditional logic was followed:

```

1 data['reviews per month'] = np.where(
2     (data['reviews per month'].isna()) & (data['number of reviews'] == 0
3     ),
4     0,
5     data['reviews per month']
6 )
7 remaining_median = data['reviews per month'].median()
8 data['reviews per month'] = data['reviews per month'].fillna(
9     remaining_median)

```

For the “last review” feature, it contained full date which was not necessary for the analysis. Users care more about the days since last review rather than the actual date.

It was transformed using a fixed reference date (2022):

```

1 reference_date = pd.Timestamp("2022-09-11")
2
3 data["days_since_last_review"] = (reference_date - data["last review"]).
  dt.days
4 data.drop(columns=["last review"], inplace=True)
5
6 data['days_since_last_review'] = data['days_since_last_review'].fillna(
7     data['days_since_last_review'].median()
8 )

```

For location coordinate features (lat, long), missing values were dropped since imputing them would create invalid geographic points. Only about 8 rows were affected.

```

1 data.dropna(subset=['lat', 'long'], inplace=True)

```

For remaining features:

Categorical features were imputed using the mode:

```

1 data["neighbourhood"] = data["neighbourhood"].fillna(
2     data["neighbourhood"].mode()[0]
3 )
4
5 data["neighbourhood group"] = data["neighbourhood group"].fillna(
6     data["neighbourhood group"].mode()[0]
7 )
8
9 data["instant_bookable"] = data["instant_bookable"].fillna(
10    data["instant_bookable"].mode()[0]
11 )
12
13 data["cancellation_policy"] = data["cancellation_policy"].fillna(
14    data["cancellation_policy"].mode()[0]
15 )

```

Numerical features were imputed using the median (less sensitive to outliers):

```

1 data["price"] = data["price"].fillna(data["price"].median())
2
3 data["service fee"] = data["service fee"].fillna(
4     data["service fee"].median()
5 )
6
7 data['minimum nights'] = data['minimum nights'].fillna(
8     data['minimum nights'].median()
9 )
10
11 data["review rate number"] = data["review rate number"].fillna(
12    data["review rate number"].median()
13 )
14
15 data["calculated host listings count"] = data[
16    "calculated host listings count"
17 ].fillna(data["calculated host listings count"].median())

```

7.8 Feature Engineering

A new categorical feature “stay category” was created instead of using raw minimum nights: Short-stay (1–3), Weekly (4–7), Monthly (8–30), Long-term (30+).

```

1 bins = [0, 3, 7, 30, 365]
2 labels = ['Short-stay (1-3 nights)',
3           'Weekly (4-7)',
4           'Monthly (8-30)',
5           'Long-term (30+)']
6
7 data['stay_category'] = pd.cut(
8     data['minimum nights'],
9     bins=bins,
10    labels=labels
11 )

```

A new feature called listing status was created based on recency of last review:

```

1 def categorize_activity(days):
2     if days <= 365:
3         return 'Recently Active'
4     elif days <= 1095:
5         return 'Stale (1-3 years)'
6     else:
7         return 'Inactive'
8
9 data['listing_status'] = data['days_since_last_review'].apply(
    categorize_activity)

```

Creation of interaction features:

```

1 data['est_monthly_revenue'] = data['price'] * data['reviews per month']
2 data['popularity_score'] = data['reviews per month'] * data['review rate
    number']
3 data['listing_age'] = 2022 - data['Construction year']

```

7.9 Outlier Handling

A boxplot was used to identify outliers.

Removed outliers indicating invalid values:

```

1 data = data[data['days_since_last_review'] > 0]
2 data['days_since_last_review'] = data['days_since_last_review'].clip(
    upper=2000)

```

Geographic filtering:

This approach filters the dataset to keep only observations whose coordinates fall within the designated latitude and longitude bounds. Any data points located outside this geographic range—which are considered spatial outliers—are excluded from the analysis.

```

1 data = data[data['lat'].between(40.5, 40.95) & data['long'].between(-74.
    3, -73.7)]

```

Minimum nights capped at 365:

For the `minimum_nights` feature, the analysis focuses on stays of at most one year. Therefore, clipping was applied to cap the maximum value at 365 nights, ensuring that any listing requiring a minimum stay longer than one year is treated as a 365-day maximum for the purpose of this analysis.

```

1 data['minimum_nights'] = data['minimum_nights'].clip(upper=365)

```

For skewed features, log transformation was applied:

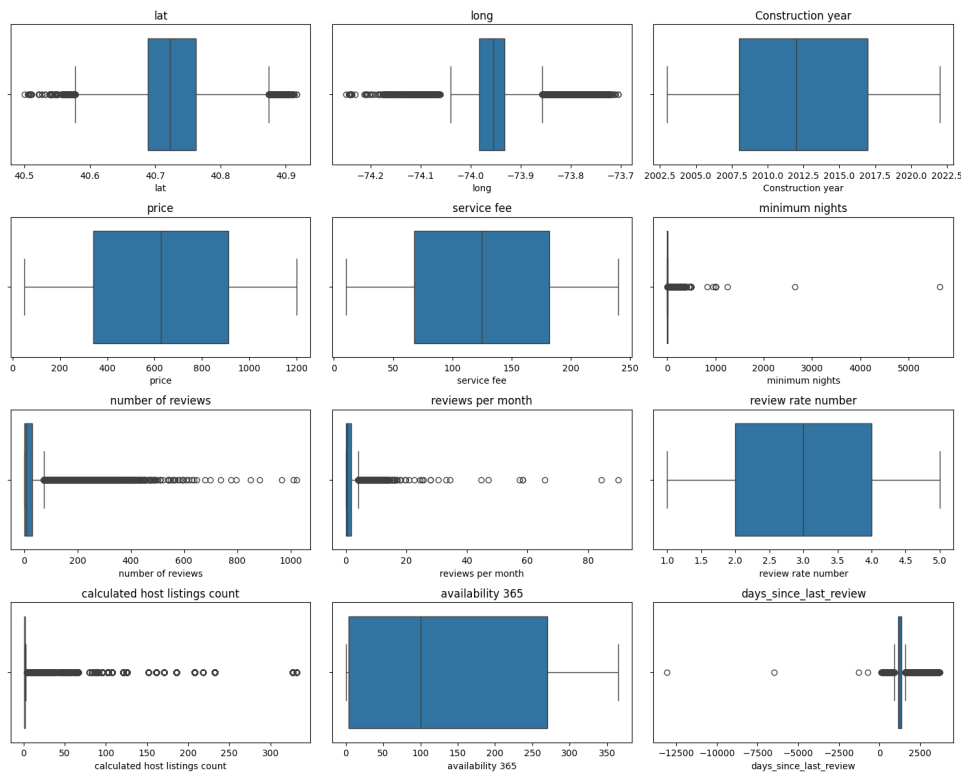


Figure 17: boxplot for outlier detection

```

1 skewed_cols = ['number of reviews', 'reviews per month', 'calculated
  host listings count']
2
3 for col in skewed_cols:
4     data[f'{col}_log'] = np.log1p(data[col])

```

7.10 Objective Analysis

Revenue estimation correlation matrix:

```

1 revenue_factors_list = ['est_monthly_revenue', 'price', 'reviews per
  month', 'review rate number', 'minimum nights', 'listing_age', '
  service fee']
2
3 revenue_correlation = data[revenue_factors_list].corr()
4
5 revenue_correlation['est_monthly_revenue'].sort_values(ascending=False)

```

Location analysis:

```

1 data.groupby('neighbourhood_group')['est_monthly_revenue'].mean()
2 data.groupby('neighbourhood')['est_monthly_revenue'].mean().head(10)

```

Demand analysis:

```

1 data.groupby('minimum_nights')['reviews per month'].mean()
2 data.groupby('cancellation_policy')['reviews per month'].mean()

```

Listing optimization:

```

1 data.groupby('instant_bookable')['est_monthly_revenue'].mean()

```

```
2 data.groupby('review_rate_number')['est_monthly_revenue'].mean()
```

Market segmentation:

```
1 data.groupby('room_type')[['price','reviews_per_month','  
est_monthly_revenue']].mean()
```

Listing age analysis:

```
1 data['age_bracket'] = pd.cut(  
2     data['listing_age'],  
3     bins=[0, 5, 10, 15, 20, 25],  
4     labels=['New (0-5)', 'Modern (6-10)', 'Standard (11-15)', 'Old  
5     (16-20)', 'Vintage (21+)'])
```

Host and availability impact:

```
1 data.groupby('host_identity_verified')['est_monthly_revenue'].mean()  
2  
3 data['availability_type'] = data['availability_365'].apply(  
4     lambda x: 'High Availability' if x > 200 else 'Seasonal/Low'  
5 )  
6  
7 data.groupby('availability_type')['est_monthly_revenue'].mean()
```

Host type comparison:

```
1 data['host_type'] = data['calculated_host_listings_count'].apply(  
2     lambda x: 'Professional' if x > 1 else 'Individual'  
3 )  
4  
5 data.groupby('host_type')['est_monthly_revenue'].mean()
```

7.11 Visualization

All visualization figures are discussed in the illustrated figures section.

8 Professional Project Summary Table

The following table is included as an example of a clean and professional academic table suitable for a Data Engineering and Analytics project report. It summarises the essential elements of the proposed system in a concise way.

Table 1: **Professional Summary of the Data Engineering and Analytics Project**

Aspect	Description
Project Domain	Transitioning from Intuitive Guesswork to a Profitability Driven Data Analysis Approach for Landlords and Investors by examining the Overcrowding Short-Term Rental Market in New York City.
Data Sources	The "Airbnb Open Data" Dataset: An extremely large and publicly accessible dataset that has many details about each rental listing available in NYC, such as geographical locations, host traits, room types, pricing, and booking rules.
Engineering Workflow	End-to-End Batch Processing Ingestion Pipeline: Consists of structured layers: Raw Layer (source file landing), Cleaned Layer (schema standardization, string manipulation, and data type conversion), and Analytical Layer (Python groupby() aggregations)
Data Quality Issues	Encountered missing values, unstandardized text-based prices (corrupted string formats with dollar signs and commas), and temporal gaps in host and property attributes.
Storage or Modelling Approach	Multi-Tier Logical Architecture: Organized into a clear three-layer structure (Raw, Cleaned, Analytical). The domain is logically partitioned into three core entities: Host, Property/Listing, and Financial Constraints, completely isolating business logic from raw anomalies.
Implementation Tools	Built entirely using Python, Pandas (for batch extraction and structural data manipulation), Google Colab (as the interactive development environment), and Overleaf/LaTeX (for formal academic reporting).
Validation Method	Data Harmonization and Type Constraints: The pipeline is validated by executing strict data-type constraints, systematically converting string objects (like prices) into numeric floats, and handling missing values uniformly across features.
Main Contribution	Operational Value-Driven Pipeline: Demonstrates how raw, dirty transactional data can be engineered into predictive assets. It builds a crucial target metric (Estimated Monthly Revenue) and behavior features (<code>host_type</code> and <code>listing_age</code>), proving statistically that professional management yields significantly higher revenue (\$685.19 vs. \$397.27).

9 Testing and Results

The raw Airbnb open data needed testing and validation to make sure that the data featuring, transformation and visualization are accurate.

So firstly, we used `'data.shape'` to check that all the data rows and columns were uploaded. And to know the data type and the count of non-null values we used `'data.info()'`, we also

used `'data.describe()'` to know if there is a missing values and to understand the data more like whether it has outliers or not and if the data vary a lot or not.

Converting the string (values that use currency) like price and service fees into numeric values to help with the analysis and data featuring and visualization.

Handling the invalid entries in the “minimum night” column to make sure that there are no negative values, the same for “availability 365” column but we also handled the values higher than 365 days.

Missing values were handled using specific strategies depending on the data and its type, for the numerical values we used the median and for the categorical values we used the mode. Duplicates were handled, Ineffectual columns were dropped. The “neighborhood group” column entries were unified.

In the end, after every testing procedure and a feature engineering process a validation step was made to make sure we have an accurate results.

10 Discussion

The analysis demonstrates that Airbnb revenue and demand are influenced more by booking activity, location strategy, and guest convenience than by premium pricing or restrictive listing controls. Short-term stays dominate the market, indicating that guests strongly prefer flexible and lower-commitment bookings. Listings with lower minimum night requirements consistently achieve stronger occupancy and higher revenue performance.

Guest behavior also reflects a preference for listings with fewer restrictions. Properties without excessive house rules generate a larger share of revenue, suggesting that reducing booking friction improves booking conversion and guest confidence. In contrast, features commonly assumed to significantly affect demand — such as cancellation policy strictness and instant booking — show minimal impact on overall booking performance. This indicates that hosts can configure these settings according to operational preference rather than relying on them as major demand drivers.

Location remains a major determinant of profitability; however, revenue performance is concentrated within specific neighborhoods rather than evenly distributed across entire boroughs. Staten Island and selected areas in Queens outperform many traditionally popular regions, while Manhattan shows inconsistent revenue performance despite its popularity. This highlights the importance of targeting high-performing neighborhoods rather than depending on broad geographic assumptions.

Room type analysis further reveals that hotel rooms consistently generate the highest demand due to their convenience and familiarity, while private rooms maintain stable secondary demand. Entire homes and shared rooms continue to serve valuable niche segments with different traveler priorities, including privacy, group accommodation, and social interaction. Consequently, room types should be marketed strategically according to their target audience rather than treated uniformly.

One of the strongest findings from the analysis is the impact of review activity on revenue performance. Revenue growth is driven primarily by consistent booking activity rather than simply increasing nightly prices. Listings with frequent guest turnover and high review activity consistently outperform expensive listings with lower occupancy levels. This demonstrates that maintaining active bookings, competitive pricing, fast response times, and strong guest engagement strategies are more effective than focusing solely on premium pricing.

Finally, the correlation analysis confirms that booking frequency is the strongest predictor of revenue performance. Reviews per month, number of reviews, room type, and neighborhood characteristics influence revenue far more than review scores or luxury pricing alone. Longer minimum stay requirements negatively affect performance, reinforcing the importance of flexibility within the short-term rental market.

10.1 Final Strategic Decisions

- Prioritize short-term stay strategies with lower minimum night requirements to maximize occupancy and revenue.
- Reduce unnecessary house rules in order to minimize booking friction and improve guest conversion rates.
- Configure cancellation policies and instant booking settings based on operational preference rather than expected demand increases.
- Focus investment, marketing, and operational efforts on high-performing neighborhoods rather than broad borough-level targeting.
- Maintain a diverse range of room types while tailoring marketing strategies toward the specific traveler segments each room type attracts.
- Prioritize occupancy consistency, review generation, and guest engagement over aggressive pricing strategies.
- Improve operational responsiveness, maintain availability, and encourage guest interaction to sustain continuous booking activity.

11 Conclusion

This project represents complete data engineering and analytic pipeline. We went through all data engineering procedures from data acquisition, cleaning, preprocessing, feature engineering, transformation and visualization.

We focused on understanding the data, its problem and the analytical problems so we can have a better vision and understanding of the work pipeline to reach an accurate and effective results that can support the business decision.